

Transcripción de operadores

Clara Rodríguez, Alejandro Hernández y Lucía Martínez

28 de septiembre de 2025

Resumen

Este documento está destinado para la transcripción de operadores C a su equivalente en lenguaje circom.

Guía de contenidos

1. Operadores Lógicos
2. Operadores de Comparación
3. Operadores Aritméticos
4. Acceso a Arrays
5. Expresiones condicionales
6. Operadores de desplazamiento

1. Operadores Lógicos

C	Circom	Comentario
<code>!a</code>	$1 - a$	Si a es condición, su negación se consigue con la diferencia entre 1 y ésta
<code>a && b</code>	$a * b$	Si a y b son verdaderos, el resultado de su producto será 1, de lo contrario será 0
<code>a b</code>	$1 - (1-a)*(1-b)$ ó $a+b - (a*b)$	Si a, b o ambos son verdaderos, el resultado de su producto será 1, de lo contrario será 0

2. Operadores de Comparación

Un detalle a observar en las funciones *LessThan()* y *LessThanEq()* es que para números negativos podría presentar problemas.

El funcionamiento de *Num2Bits(n)* cuando recibe un número negativo es el siguiente:

- El número negativo se interpreta como un cuerpo finito.
- Éste se interpretará como un número superior a $2^n - 1$, saliéndose fuera del rango $[0, 2^n - 1]$, por tanto la constraints no se pueden cumplir y la prueba no se generará.
- En conclusión, con *Num2Bits()* garantizamos el uso de número positivos dentro del rango mencionado de manera implícita.

Observación: Gracias a la constraint `lc1 == in` (fuerza a que la entrada cumpla el rango $[0, 2^n - 1]$), podemos evitar el uso de número negativos, ya que estos no cumplirán tal restricción.

Num2Bits(n):

```

1  template Num2Bits(n){
2      signal input in;
3      signal output out[n];
4      var lc1=0; // Reconstruye el numero decimal a convertir para la
           constraint
5
6      var e2=1; // Evitar la potencia
7      for (var i = 0; i<n; i++) {
8          out[i] <-- (in >> i) & 1; // Construimos el numero en binario
9          out[i] * (out[i] -1 ) == 0;
10         lc1 += out[i] * e2;
11         e2 = e2+e2;
12     }
13
14     lc1 == in; // Constraint mencionada anteriormente
15 }

```

Partiendo de esta información, podemos realizar pequeñas modificaciones en las funciones *LessThan()* y *LessThanEq()* para asegurar el uso de enteros positivos.

<pre> 1 template SecureLessThan(n){ 2 signal input in[2]; 3 signal output out; 4 5 //----- 6 7 component checkA = 8 Num2Bits(n); 9 checkA.in <= in[0]; 10 11 component checkB = 12 Num2Bits(n); 13 checkB.in <= in[1]; 14 15 adder = BinSum(n, 2); 16 17 var i; 18 for (i=0;i<n;i++) { 19 checkA.out[i] ==> adder 20 in[0][i]; 21 checkB.out[i] ==> adder 22 in[1][i]; 23 } 24 25 // Miramos el bit + 26 significativo (MSB), 1 27 si A < B, 0 de lo 28 contrario 29 adder.out[n-1] ==> out; 30 } </pre>	<pre> 1 template SecureLessThanEq(n) 2 { 3 signal input in[2]; 4 signal output out; 5 6 //----- 7 8 component checkA = 9 Num2Bits(n); 10 checkA.in <= in[0]; 11 12 component checkB = 13 Num2Bits(n); 14 checkB.in <= in[1]; 15 16 adder = BinSum(n, 2); 17 18 var i; 19 for (i=0;i<n;i++) { 20 checkA.out[i] ==> adder. 21 in[0][i]; 22 checkB.out[i] ==> adder. 23 in[1][i]; 24 } 25 26 // Miramos el bit + 27 significativo (MSB), 1 28 si A < B, 0 de lo 29 contrario 30 component equal = IsEqual 31 (); 32 equal.in[0] <= in[0]; 33 equal.in[1] <= in[1]; 34 35 out <= adder.out[n-1] + 36 equal.out - adder.out[37 n-1] * equal.out; 38 } </pre>
--	--

Con esta pequeña modificación, al usar para ambos número a comparar la función *Num2Bits()* nos

aseguramos de que se cumpla la restricción de ser número enteros positivos dentro del rango.

C	Circom	Comentario
$a == b$	<code>IsZero()(a - b)</code>	Si $a = b$, entonces se cumple ($==1$)
$a != b$	<code>1- IsZero()(a - b)</code>	Si $a \neq b$, entonces es falso ($==0$), por tanto se invierte con la resta
$a < b$	<code>SecureLessThan(n)(a,b)</code>	Si $a < b$ es verdadero, entonces se cumple ($==1$)
$a \leq b$	<code>SecureLessThanEq(n)(a,b)</code>	Si $a \leq b$ es verdadero, entonces se cumple ($==1$)
$a > b$	<code>1- SecureLessThanEq(n)(a,b)</code>	Si $a \leq b$ es falso ($== 0$), entonces $a > b$
$a \geq b$	<code>1- SecureLessThan(n)(a,b)</code>	Si $a < b$ es falso ($== 0$), entonces $a \geq b$

Nota: Las funciones `IsZero()` son circuitos que comparan si un valor es igual a 0.

Nota: Las funciones `IsEqual()` comprueba si dados dos números, estos son iguales o no.

3. Operadores Aritméticos

Tanto en C como en Circom, los operadores suma (+), resta (-) y multiplicación (*) no difieren. En cambio, en operaciones como la división entera (/) y el módulo (%) si que encontramos diferencias y por ende es necesaria su transcripción.

La idea desde la que se parte para encontrar esta equivalencia es de la propiedad fundamental de la división entera, la cual establece:

$$a = b \cdot q + r$$

Operación división

Partiendo de la fórmula $a = b \cdot q + r$, podemos llegar al resultado de la operación a/b forzando a que se cumpla la fórmula pero despreciando el resto (ya que se trata de división entera).

El término "forzar" hace referencia a usar una variable en Circom, de forma que partiendo del dividendo y del divisor, el cociente solo pueda tener ese valor.

Ejemplo de código en Circom:

```

1      template IntegerDivision() {
2          signal input a; // Dividendo
3          signal input b; // Divisor
4          signal output c; // Cociente q
5
6          var q;
7          a === b * q;
8          c <= q;
9      }
10
11      component main = IntegerDivision();

```

Operación módulo

Partiendo de la fórmula $a = b \cdot q + r$, podemos llegar al resultado de la operación $a \% b$ con la variable r, la que representa el resto de una división.

Por otro lado, ha de cumplirse la expresión $0 \leq r < b$. De lo contrario, podría producirse problemas en cuanto a la seguridad del circuito.

$$a = b \cdot q + r \quad (1)$$

$$r = a - (b \cdot q) \quad (2)$$

$$r = a - (b \cdot (a/b)) \quad (3)$$

Nota: q representa el resultado de una división, por lo que podemos decir que $q = a/b$.

Para que se cumpla la restricción $0 \leq r < b$:

```

1      component lessThan = SecureLessThan(n);
2      lessThan.in[0] <== r;
3      lessThan.in[1] <== b;
4
5      // 1 si r < b, como ya se checkea en SecureLessThan que sea positivo o 0
        es suficiente
6
7      lessThan.out == 1 // Hacemos que se cumpla la restriccion,
8      // Si no se cumple, no se generara la prueba

```

C	Circom	Comentario
a/b	$a == b * q + r$	Forzando q con una variable a que esa ecuación se cumpla
$a \% b$	$a - (b * (a/b))$	a/b hace referencia a la equivalencia que se obtiene en circom para poder realizar la operación %

4. Acceso a Arrays

La diferencia principal entre C y Circom en el acceso a arrays radica en que el tamaño de los arrays en Circom ha de ser fijo a la hora de compilar y ser siempre constante, mientras que en C puede ser estático y dinámico. Además, en Circom el valor de los arrays no puede ser reasignado dinámicamente.

Nota: En C, el tamaño de un array puede ser variable, ya sea por arrays de longitud variable o por el uso de memoria dinámica

<pre> 1 //Codigo en C 2 #include <stdio.h> 3 4 int main() { 5 6 // Ejemplo de array con tamaño variable con funcion f() 7 int n = f(); 8 int lista[n]; // Se decide en tiempo de ejecucion 9 10 11 // EJemplo de array de tamaño variable con asignacion de memoria dinamica 12 int *lista = malloc(x); 13 14 //x tamaño que se quiera reservar en memoria dinamica 15 } </pre>	<pre> 1 2 //Codigo circom 3 template accesoArray(N) { 4 signal input lista[N]; // Tamaño fijo, suponemos iniciada 5 signal output sumaAcumulada; 6 7 var suma = 0; 8 for(int i = 0; i < N; i++){ 9 suma = suma + lista[i]; 10 11 // NO se puede hacer lo siguiente 12 // lista[i] <== 3; 13 } 14 15 sumaAcumulada <== suma; 16 } 17 18 component main = accesoArray 19 (10); </pre>
--	--

Nota: En Circom no se puede fijar el tamaño de un array con una señal.

En Circom, no se puede acceder directamente a posiciones de arrays con señales inputs. Sin embargo, se puede acceder indirectamente a esta posición sin usar esta señal input.

```

1      signal input pos;
2      signal input lista[N];
3      signal output acceso;
4
5      // NO SE PUEDE HACER
6      //lista[pos];
7
8      // SI SE PUEDE HACER, coste O(N)
9      // Hay otras formas como arboles que reducen el coste a O(log (N))
10     for(int i = 0; i < N; i++){
11         signal seleccionado;
12         seleccionado <== (i === pos ? 1 : 0);
13         acceso <== acceso + lista[i] * seleccionado;
14     }

```

5. Expresiones condicionales

En C, al ser un lenguaje de ejecución dinámica, los condicionales pueden decidir el valor final de una variable en tiempo de ejecución.

En Circom, cada señal representa un “cable” del circuito, que solo puede recibir un valor. Por ello, no es posible reasignar un señal dentro de un condicional dinámico.

Para simular decisiones condicionales basadas en entradas (inputs), se deben usar expresiones aritméticas que implementen la lógica de selección, de manera que el valor de salida quede correctamente definido sin reasignar el signal.

<pre> 1 //Codigo en C 2 #include <stdio.h> 3 4 int main(int argc, char * 5 argv[]) { 6 int entrada = argv[1]; 7 char mensaje[50]; 8 9 if(entrada % 2 == 0){ 10 sprintf(mensaje, "%d es 11 par\n", entrada); 12 } 13 else { 14 sprintf(mensaje, "%d es 15 impar\n", entrada); 16 } 17 18 printf("s", mensaje); 19 free(mensaje); 20 21 return 0; 22 } </pre>	<pre> 1 //Codigo circom 2 template condicionales() { 3 signal input entrada; 4 signal output salida; //0 5 par, 1 impar 6 7 var var_division; 8 entrada == 2 * 9 var_division; 10 11 salida <== (entrada - (2* 12 var_division)) 13 } 14 15 component main = 16 condicionales(); </pre>
--	---

Con este ejemplo se ilustra como puede transformarse una expresión condicional que comprueba si un número es par o impar en una expresión aritmética que representa lo mismo y evita el reasignar la señal de salida más de una vez (algo no permitido).

6. Operadores de desplazamiento

Los operadores de desplazamiento en C funcionan de la siguiente manera:

Imaginemos que tenemos el número 2 que queremos desplazar 2 bits

- 1. Dado un número decimal 2, transformarlo a binario, que es 0010
- 2. Con el n (número de bits a desplazar), desplazamos los dos últimos bits
- 1000 (4 en decimal), es el número que se obtiene desplazando los bits

*Observación: Podemos observar que para obtener el número equivalente del desplazamiento de bits en decimal se consigue con $x * 2^n$ en el caso de desplazamiento a la derecha y $x * 2^n$ en el caso de desplazamiento a la izquierda.*

Podemos hacer uso de esta observación para llegar a la equivalencia de tal operación en Circom.

C	Circom
$a \ll b$ 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17	<pre> //Codigo circom template desplazamiento_izq(N) { signal input numero_decimal; signal output salida; var potencia = 1; for (var i = 0; i < N; i++) { potencia = potencia * 2; } salida <== numero_decimal * potencia; } component main = desplazamiento_izq(N); </pre>
$a \gg b$ 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21	<pre> //Codigo circom template desplazamiento_dcha(N) { signal input numero_decimal; signal output salida; var potencia = 1; for (var i = 0; i < N; i++) { potencia = potencia * 2; } var division; numero_decimal === potencia * division; salida <== division; } component main = desplazamiento_dcha(N); </pre>